





Practical Computer Vision on Edge Devices

Asst. PhD(s) Andrei Radu
Lect. PhD. Georgian Nicolae

Multimedia Technologies in Biometrics and Information Security Applications
National University of Science and Technology ''Politehnica" of Bucharest

October 2026

Content

1. Image Recognition
2. Object Tracking
3. Person Segmentation

1. Image Recognition

Rock Paper Scissors

Why Gestures ?

- The recognized hand gestures $\rightsquigarrow$ **game moves** for human player.
- Gesture recognition enhances human-computer interaction in various applications, especially gaming.

Rock Paper Scissors

Why Gestures ?

- The recognized hand gestures $\rightsquigarrow$ **game moves** for human player.
- Gesture recognition enhances human-computer interaction in various applications, especially gaming.



Figure 1: Rules of Rock Paper Scissors Lizard Spock

Used Dataset

HaGRID

- HaGRID (HAnd Gesture Recognition Image Dataset) contains **553,991** FullHD RGB images divided into **18 classes** of gestures.
- Used for hand detection, keypoints recognition and **hand gesture recognition**.



Figure 2: HaGRID Logo, inspired by the Harry Potter character: Rubeus Hagrid

Reference: Kapitanov et al., "HaGRID - HAnd Gesture Recognition Image Dataset", arXiv 2022.

Gestures and Moves



Figure 3: Classes of HaGRID. The term *inv* stands for *inverted*.

Gesture to Move

- Rock: fist
- Paper: stop & stop inverted
- Scissors: two up, two up inverted, peace & peace inverted

Hand Recognition Model

Available Models

The authors proposed multiple architectures, for different tasks:
Hand Detection:

- RetinaNet (w/ ResNet-50)
- SSDLite (w/ MobileNet v3)
- Yolo (V7 tiny)

Gesture recognition (from full frame):

- ResNet (and ResNeXt)
- ViT
- MobilNet

Models Comparison

What are we looking for?

- A fast model, which will be able to run on Jetson Nano
- For computing inference time $\rightsquigarrow$ Intel(R) Xeon(R) Platinum 8168
- **What time do we have between the captured frames?**

Model	Size (MB)	Parameters (M)	Inf. time (ms)	F1
ResNet-18	89.6	11.2	49.25	97.5
ResNet-152	466.5	58.3	292.6	95.5
ResNeXt-50	184.6	23.2	135.6	**98.3**
ResNeXt-101	696.4	87	397.2	97.5
MobileNetV3 small	**12.5**	**1.6**	**10.6**	86.4
MobileNetV3 large	34	4.3	33.4	91.9
VitB16	686.6	85.9	325.5	91.1

Table 1: Model comparison for the full-frame detectors

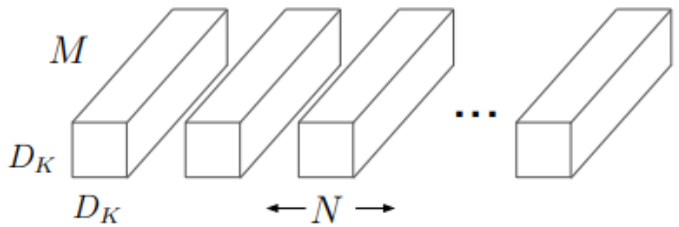
MobilNets

MobilNet v1

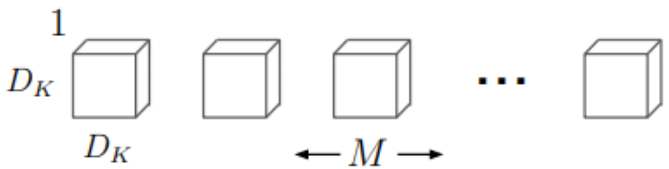
MobilNet introduces the concept of **Depthwise convolution**:

- Filters are applied *independently* to each input channel, reducing the computation.
- Instead of convolving all channels together, each channel has its own 2D convolution filter.

$$\hat{G}_{k,l,m} = \sum_{i,j} \hat{K}_{i,j,m} \cdot F_{k+i-1,l+j-1,m}$$



(a) Standard Convolution Filters



(b) Depthwise Convolutional Filters

Figure 4: Convolution and Depthwise Convolution filters

Reference: Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications", arXiv 2017.

MobilNets

MobilNet v2

The second version introduces the **Inverted Residual block**

- Flips the traditional structure: **expands** the input dimensions first, applies depthwise convolutions, and then **projects back** to a smaller dimension.
- The input is first expanded to a higher-dimensional feature space, followed by pointwise and depthwise convolutions.

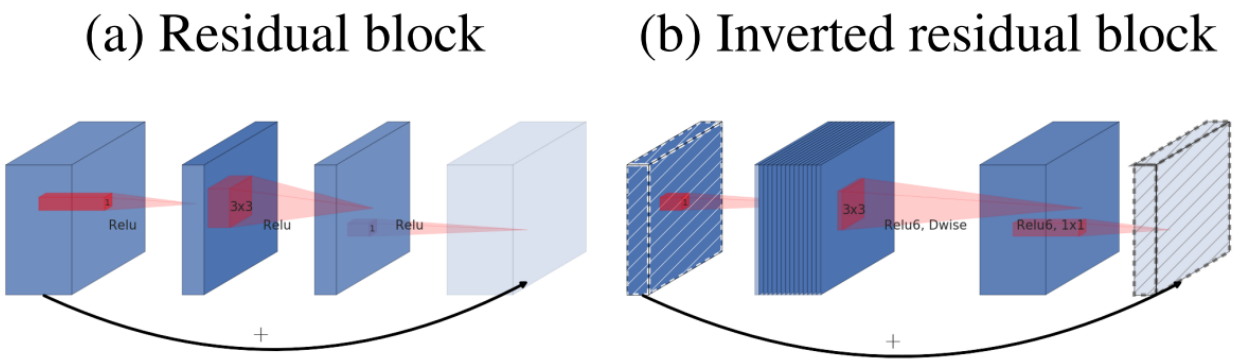


Figure 5: Residual block and Inverted Residual block

Reference: Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks", arXiv 2019.

MobilNets

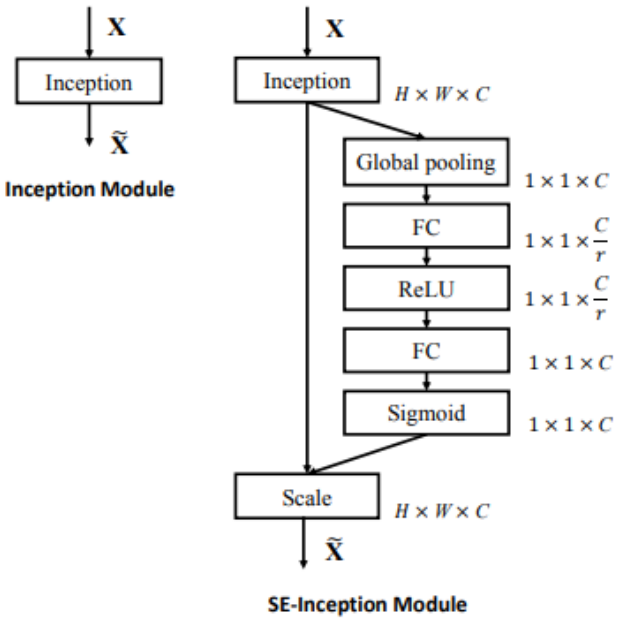


Figure 6: Squeeze and Excitation added to an Inception block

MobilNet v3

Adds the Squeeze-and-Excitation (SE):
Squeeze Operation: Global Average Pooling is applied to each channel to generate a channel descriptor (a single value per channel). This "squeezes" the spatial dimensions into a single representative value.
Excitation Operation: The channel descriptor is passed through fully connected layers and a non-linear activation to generate channel-wise weights. These weights are multiplied back with the original feature map, thus "exciting" the most important channels.

ONNX Model Format

What is ONNX?

- ONNX (*Open Neural Network Exchange*) is an open-source format that facilitates interoperability between different ML frameworks.
- Supports models trained in frameworks like PyTorch, TensorFlow and more.

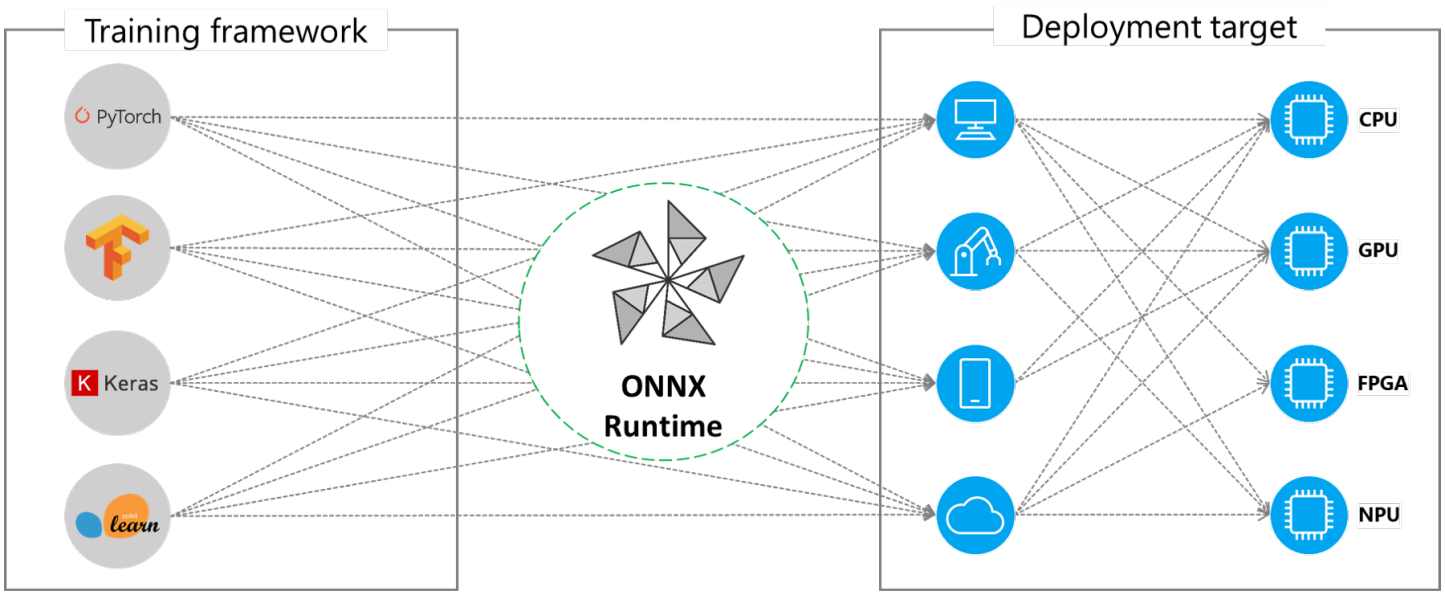


Figure 7: ONNX Operability

ONNX Model Conversion

Why Use ONNX?

- Allows easily converting models from one framework to another.
- Enables **optimized execution** across different hardware platforms: CPUs, GPUs, and specialized accelerators.
- ONNX has a large ecosystem with support from platforms like AWS, **NVIDIA** and Intel.

Exporting Models

Exporting models to ONNX is supported by multiple frameworks:

- PyTorch is natively supported by providing a simple function call: `torch.onnx.export()`.
- TensorFlow models can be converted using `tf2onnx` or `onnx-tf`.
- Tools like `onnx-simplifier` can be used to reduce model complexity.
- We can use `onnx-runtime` to run the models without having a framework installed.

Application Flow

Restrictions

- Model accepts an input of 224px × 244px.
- We should use as much available data as possible.
- **What optimizations can we do, to make it run faster?**
- **How about the correctness of the app?**

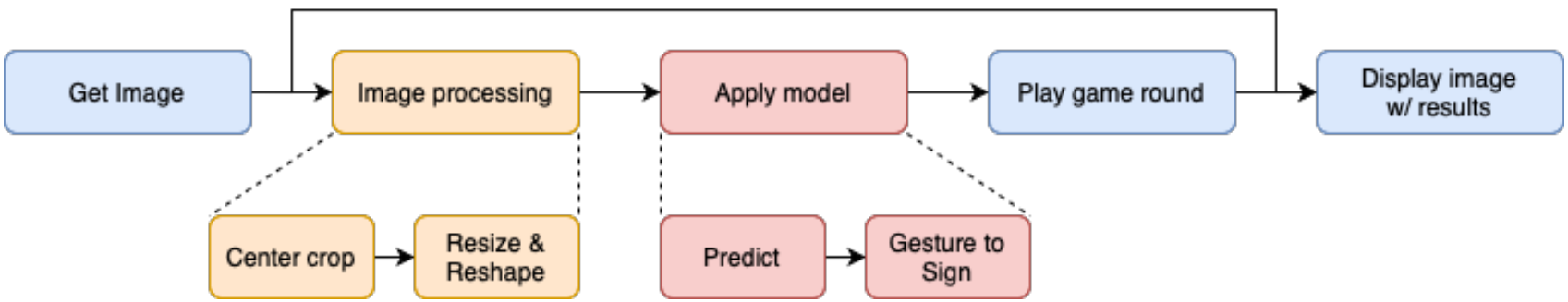


Figure 8: Application flow

Practical Part

Let's play!

- Go to the following repository: **github.com/andrei-radu/iasi-lab-cv**
- Follow the installation instructions.
- Run the first app (RPS).

Try it out

Complete the following table with the required information:

Prediction	Mean time for 1s	Speed-up
Each frame		-
3 times per s		
1 time per 1		

Practice I

Multiprocessing?

Instead of predicting based on several frames passed, dedicate a thread in which the computation is done.

- Use Python's `multiprocessing` library.
- Global variables and data structures might be useful.

Rock Paper Scissors Lizard Spock?

- Choose two (or more) other signs from the recognised ones and associate them with new moves: **lizard** and **Spock**.
- Implement the new game logic, by modifying the scripts.
- The rules for this game are presented on the `Introduction` slide.

Practice II

Better strategy?

Try to implement new strategies for the computer. These can be based on the psychology of most players, which tend to:

- uses the same move if they won the last round.
- change the move if they lost.

Two Player Game?

Change the image processing module such that:

- Create two images from the source, using the left-most and right-most parts.
- Predict the sign from each image.
- Modify the game to allow two players.

2. Object Tracking

Object Tracking with OpenCV

Tracking vs. Detection

- **Object detection:** identifies objects and their locations in an image.
- **Object tracking:** follows a selected object across successive frames.



Figure 9: Object Detection vs Tracking

Limitations

Main issues

- **Occlusion:** another object may temporarily hide the tracked object.
- **Appearance changes:** rotation, illumination, or deformation can make tracking more difficult.
- **Fast motion:** the object may move outside the tracker's search region.
- **Similar objects:** the tracker may switch to a visually similar region.
- **Tracking failure:** a tracker may lose the target and generally cannot guarantee automatic reacquisition.

Discussion

- How could we detect that tracking has failed?
- How could object detection help recover the target?

Object Trajectory and Speed

Getting the trajectory

Let the bounding box at frame t be $B_t = (x_t, y_t, w_t, h_t)$.
Its center is $c_t = \left(x_t + \frac{w_t}{2}, y_t + \frac{h_t}{2}\right)$.
Steps for tracking:

- Store the center position for every successfully tracked frame.
- Connect consecutive centers with line segments.
- The resulting trajectory visualizes the object's movement.

Estimating Motion

The displacement magnitude is

$$d_t = \|c_t - c_{t-1}\|_2.$$

Dividing by the elapsed time gives an approximate image-plane speed in pixels per second.

Application Flow

Usage

- Without tracking, the application just displays the camera input.
- To freeze the frame and select the ROI, press the `R` key.
- The selection is done by drawing a rectangle with the mouse.
- The motion trail can be cleared using the `C` key.

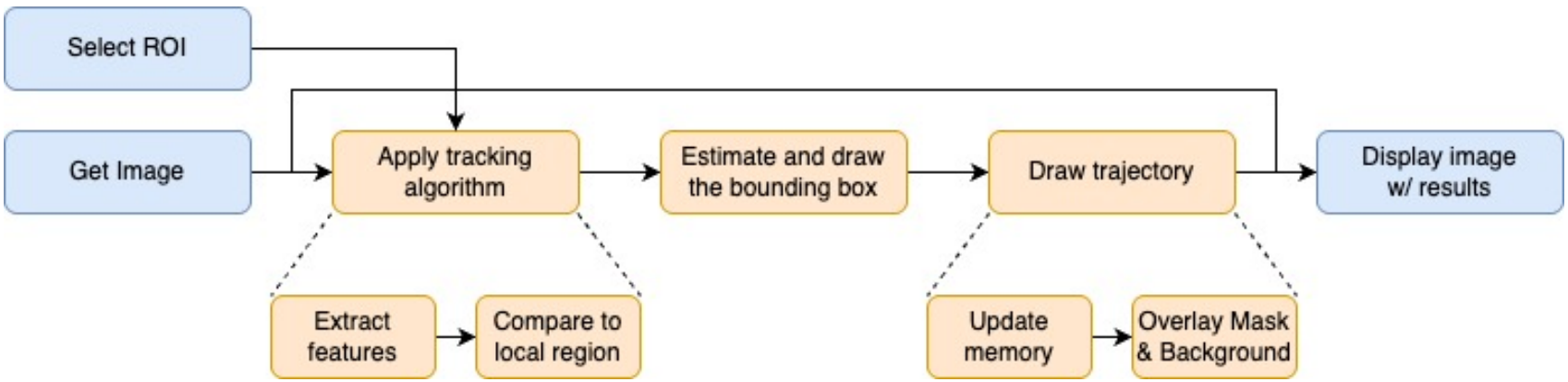


Figure 10: Application flow

KCF: Kernelized Correlation Filters

Used by default by the demo app.

Algorithm

- **Correlation filter:** Learns an appearance model from the selected object and searches for its most likely location in the next frame.
- **Kernel trick:** Uses kernel methods to model nonlinear relationships between image features without explicitly computing high-dimensional feature mappings.
- **Fast computation:** Exploits circular shifts of image features and the Fast Fourier Transform (FFT) to efficiently evaluate candidate object locations.
- **Model update:** Updates the appearance model as the object moves, helping the tracker adapt to gradual changes.
- **Main limitation:** Can struggle with significant scale changes, occlusion, and rapid appearance changes.

Reference: Henriques et al., "High-Speed Tracking with Kernelized Correlation Filters," IEEE TPAMI, 2015.

CSRT: Correlation Filter Tracking

A better (but slower) alternative: **CSRT** (Discriminative Correlation Filter with Channel and Spatial Reliability).

Algorithm

- **Correlation filter:** compares visual features in a local search region to estimate the target location.
- **Spatial reliability:** emphasizes useful spatial regions of the object instead of treating every pixel in the bounding box as equally reliable.
- **Channel reliability:** assigns different importance to feature channels according to their usefulness for localization.
- **Model update:** adapts the appearance model as the object changes between frames.

Reference: Lukezic et al., "Discriminative Correlation Filter with Channel and Spatial Reliability," CVPR 2017.

Practice I

CSRT vs. KCF?

Compare two OpenCV tracking algorithms.

- Run the same experiment using CSRT and KCF.
- Measure the average processing time per frame and FPS.
- Compare their behavior under fast motion, occlusion, and changes in illumination.

Tracking Multiple Objects?

Extend the application to track more than one object.

- Allow the user to select multiple bounding boxes.
- Create and update one tracker for each selected object.
- Display a different color and trajectory for each object.

Practice II

Predict the Next Position?

Use the object's previous positions to estimate its future location.

- Estimate the displacement from the last two positions.
- Predict the center in the next frame using constant velocity.
- Draw the predicted center and compare it with the actual one.

Automatic Object Selection?

Combine object detection and tracking.

- Use a pretrained detector to locate objects in a frame.
- Initialize a tracker from a selected detection.
- Compare the computational cost of running detection on every frame with periodically detecting and tracking between detections.

3. Person Segmentation

What is segmentation?

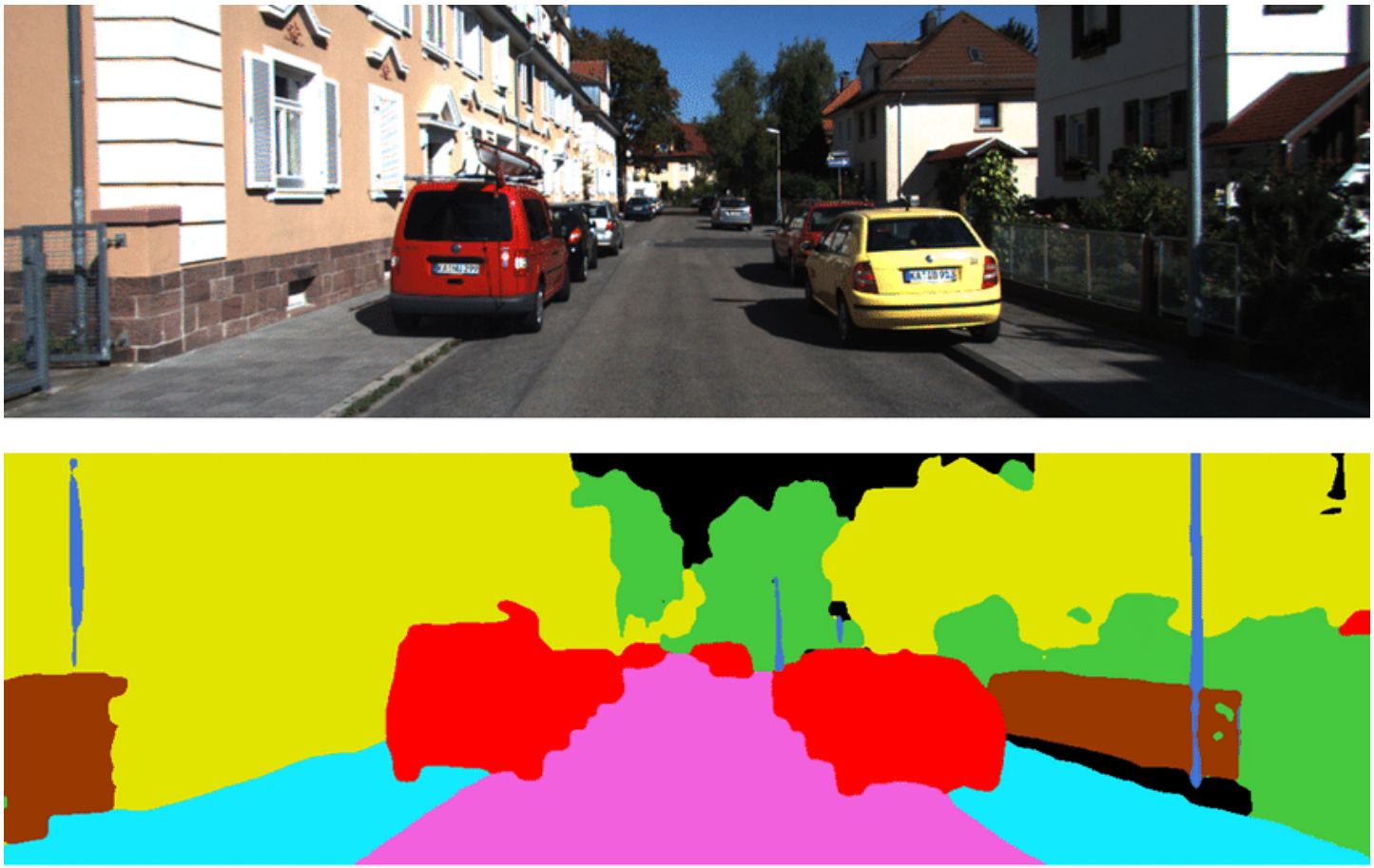


Figure 11: Example of Semantic Segmentation for Autonomous Driving

LR-ASPP: Lightweight Semantic Segmentation

Model info

- **LR-ASPP** stands for Lite Reduced Atrous Spatial Pyramid Pooling.
- It is a lightweight segmentation head designed to work with efficient feature extraction backbones.
- We can use the same MobileNetV3 to extracts visual features.
- LR-ASPP head transforms these features into **pixel-level class predictions**.
- Trained on the **Cityscapes dataset**.

Key idea

Combine high-level semantic information with spatial details from lower-level features to predict a class for every pixel.

Reference: Howard et al., "Searching for MobileNetV3," ICCV, 2019.

LR-ASPP: Architecture

Components

1. **High-level features:** Extracted from deeper backbone layers to encode semantic information and broader contextual cues.
2. **Global context branch:** Average pooling summarizes the high-level feature map using SE branches.
3. **Low-level feature fusion:** The modulated features are upsampled and combined with a 1×1 projection of lower-level features, preserving finer spatial information.
4. **Pixel classification:** A final convolution produces class logits, which are upsampled to the input resolution.

Design advantages

- Lightweight segmentation head.
- Efficient global-context aggregation.
- Fusion of semantic and spatial features.
- Low computational overhead.

LR-ASPP Architecture

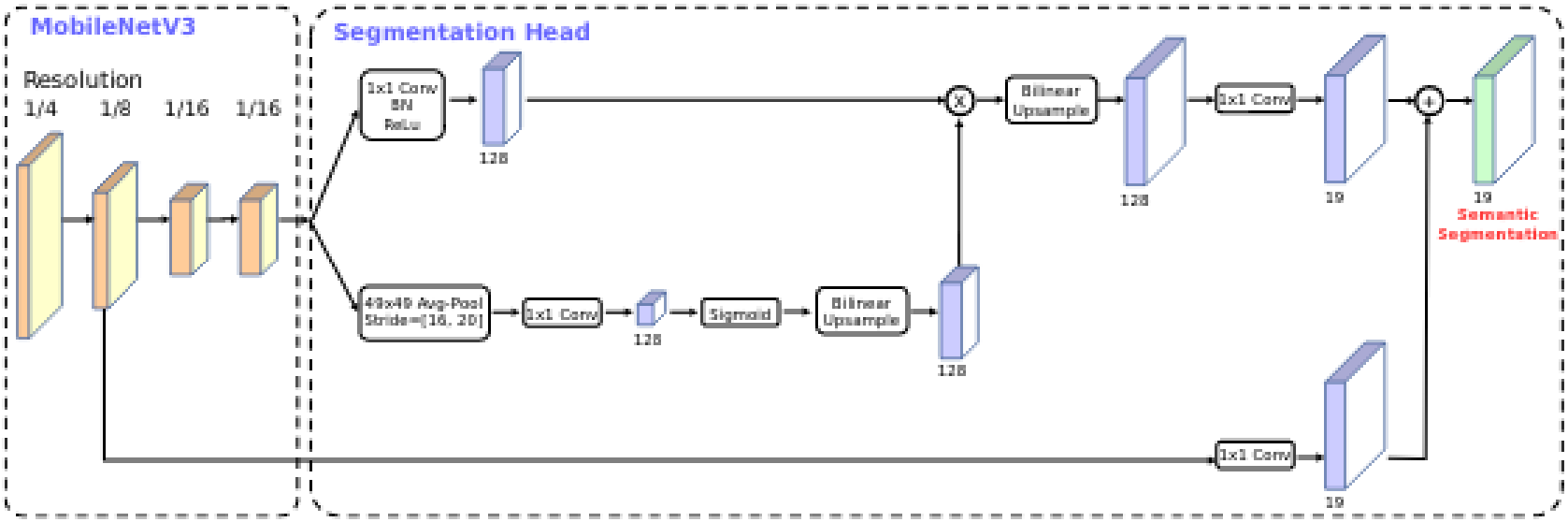


Figure 12: LRASPP with MobilNetV3

Application Flow

Restrictions

- Model accepts an input of 256px × 256px.
- The default threshold used is 50%.
- The segmentation mask is scaled back to the original image shape ⇝ blocky segmentation depending on the initial resolution.

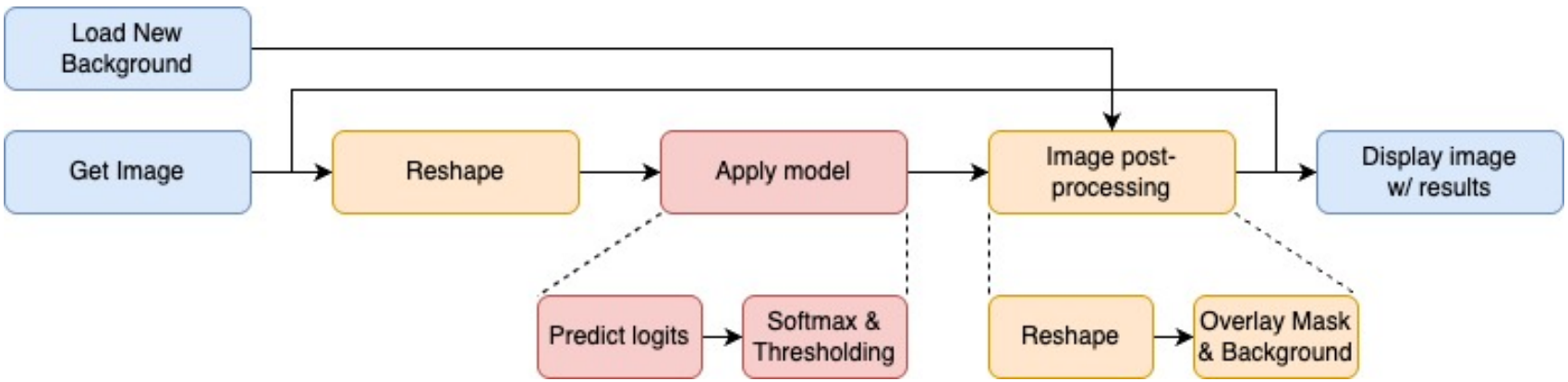


Figure 13: Application flow

Practice I

Virtual Background?

Modify the segmentation script to replace the background with an image of your choice.

- Load an image and resize it to match the camera frame.
- Use the person segmentation mask to combine the person with the new background.
- Try different backgrounds that best match the current scene.

Virtual Blur?

Modify the segmentation script to replace the background with a blurred version of the original background.

- Duplicate the image from the camera feed and blur it.
- Use the person segmentation mask to combine the person with the blurred background.
- Restrict the segmentation only to the main person in the frame.

Practice II

Better Segmentation?

Experiment with the predicted segmentation mask.

- Modify the confidence threshold used to classify pixels as belonging to a person.
- Compare the results for different threshold values.
- Investigate the trade-off between missing parts of the person and incorrectly including the background.

Temporal Stability?

Reduce flickering in the predicted segmentation mask.

- Combine the current mask with the mask from the previous frame.
- Experiment with a moving average or exponential smoothing.
- Compare the stability and responsiveness of the mask when the person moves quickly.

Thank You!